what is the best name for a new programming language I invent?

clicker.cs.illinois.edu

Q1

~Code~
340

CS 340

Program's Memory | Virtual Memory

Updates

1. MP4 UTF-8 due today

2. MP5 Allocator out today

3. HW 4 due Wednesday at midnight

4. 2nd chance exam released soon

Program's Memory | Virtual Memory

Today's LGs - Deepen your understanding of how memory works in a C program and on your computer on a whole.

Be able to differentiate between globals/static, the stack, and the heap
Be able to identify common issues with the stack and the heap use
Be able to implement a simplified, efficient version of the C allocator
Be able to articulate why we use virtual memory in a computer
Be able to put together many system concepts with virtual memory

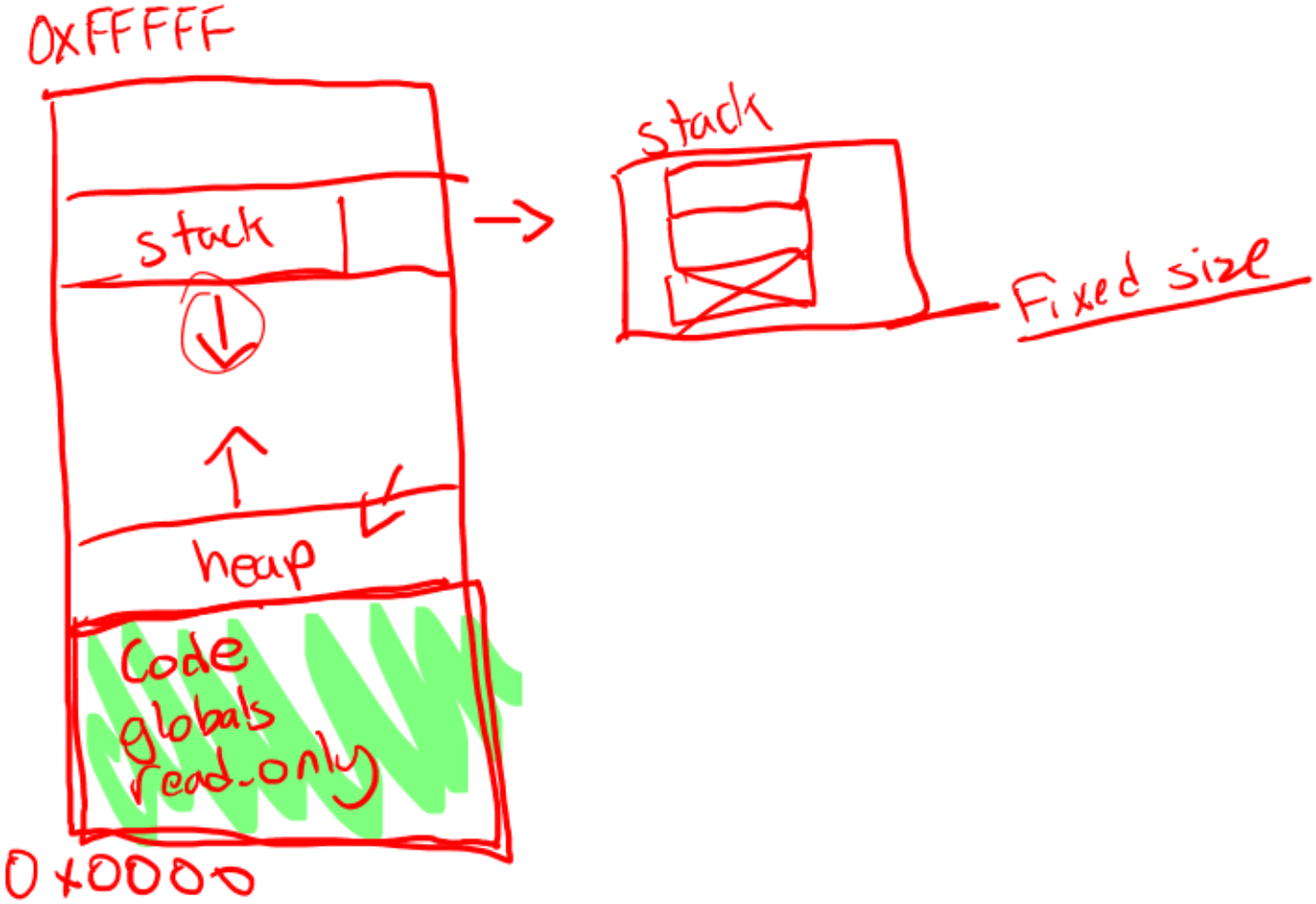
Agenda

1. A Program's Memory
   a. Description
   b. Clickers
   c. MP5 intro

2. Virtual Memory
   a. Description
   b. Examples
   c. Clickers

3. MP5 Demo

The C Program's View of its Memory Space



Globals/Static Memory

Code → assembly → machine code (1s and 0s)

Read only globals → char *str = "hello";

stack

read-only

Static and global variables

The Stack

- fixed size
- stack frames get added and removed

stack overflow – run out of stack space

use memory after function returns

The Heap

Grow upwards

Program request space on a as needed basis

```
6   int *func(int y){
7       int x = y + y + y;
8       int *ptr = &x;
9       return ptr;
10  }
11
12  int main(){
13      int *ptr = func(7);
14      printf("%d\n", *ptr);
15  }
```

PROBLEMS OUTPUT TERMINAL 3

TERMINAL

```
root@7a6ba03ac098:/workspaces/bst#
21
```

Is there an error?

clicker.cs.illinois.edu

Q2

~Code~ 340

A) Yes - undefined behavior

B) No - looks good

C) Yes - memory leak

D) Yes - stack overflow

```
6   int *func(int y){
7       int x = y + y + y;
8       int *ptr = &x;
9       return ptr;
10  }
11
12  int main(){
13      int *ptr = func(7);
14      func(1);
15      printf("%d\n", *ptr);
16  }
17
```

21

PROBLEMS OUTPUT TERMINAL 3

TERMINAL

```
root@7a6ba03ac098:/workspaces/bst#
3
```

Why does it print 3 not 21?

clicker.cs.illinois.edu

Q3

~Code~ 340

undefined!

What is true about this code?

clicker.cs.illinois.edu

Q4

~Code~
340

```
int main(){
    char *str = "hello";
    char str2[6] = "hello";

    printf("%#X and %#X\n", str, str2);
}
```

read-only

array

A) str and str2 will always hold different addresses — True

B) str will always hold a smaller address than str2 — True

C) str lives on the heap but holds the address to read only memory

prints- 0x006B38 and 0xE554520

How many bytes do str and str2 need on the stack? Assuming 8 byte addresses.

clicker.cs.illinois.edu

Q5

~Code~
340

```
int main(){
    char *str = "hello";
    char str2[6] = "hello";

    printf("%#X and %#X\n", str, str2);
}
```

str needs 8 bytes
str2 needs 6 bytes

A) 12 bytes
B) 14 bytes
C) 16 bytes
D) 18 bytes

Agenda

1. A Program's Memory
   a. Description
   b. Clickers
   c. MP5 intro

1. Virtual Memory
   a. Description
   b. Examples
   c. Clickers

1. MP5 Demo

Using the Heap in C

`malloc(size)` - allocates size bytes and returns a pointer to the allocated memory.

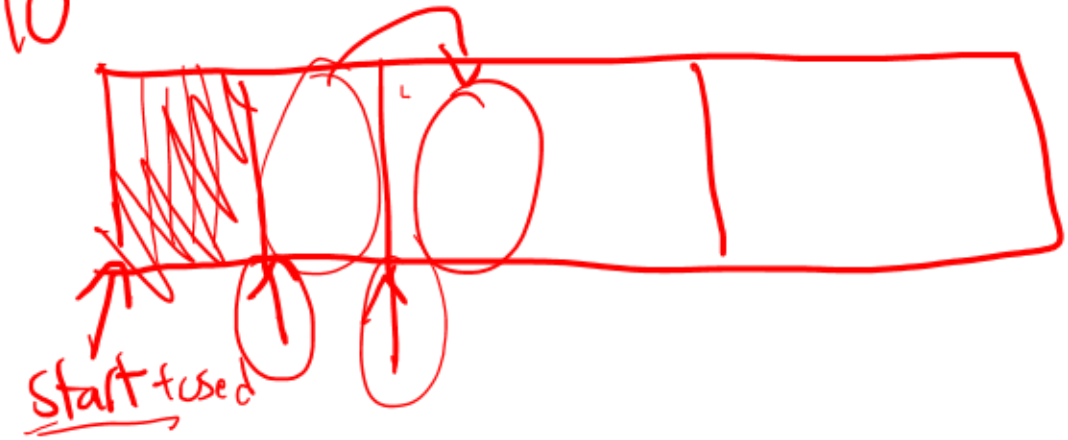
`free(p)` - frees the memory space pointed to by p which must have been allocated previously with a call to malloc.

`realloc(p, size)` - changes the size of allocated memory pointed to by p to size bytes. Contents of memory will be unchanged if within new range. If size is 0, this is the same as free(p). The function returns a pointer to the new block of allocated memory or NULL if size is 0.

MP5 - working slow/bad solution

~~malloc~~ used = ~~0~~ ~~0~~ 10

size_t



malloc(6)

malloc(4)

free()

realloc

memcpy

free

malloc

MP5 - Example

```
const char *mytest(allocator *a) {
  void *ptr = a->malloc(4);
  void *ptr2 = a->malloc(5);
  a->free(ptr);
  a->free(ptr2);
  void *ptr3 = a->malloc(4);
  if(ptr3 < ptr2){
    return "not the base algorithm";
  }
  return 0;
}
```

Takeaways

- A C program sees its addresses as starting at 0 and going until 2^64.
- C code -> assembly -> machine code (1s and 0s)
- Heap - grow up
- Stack - grows down

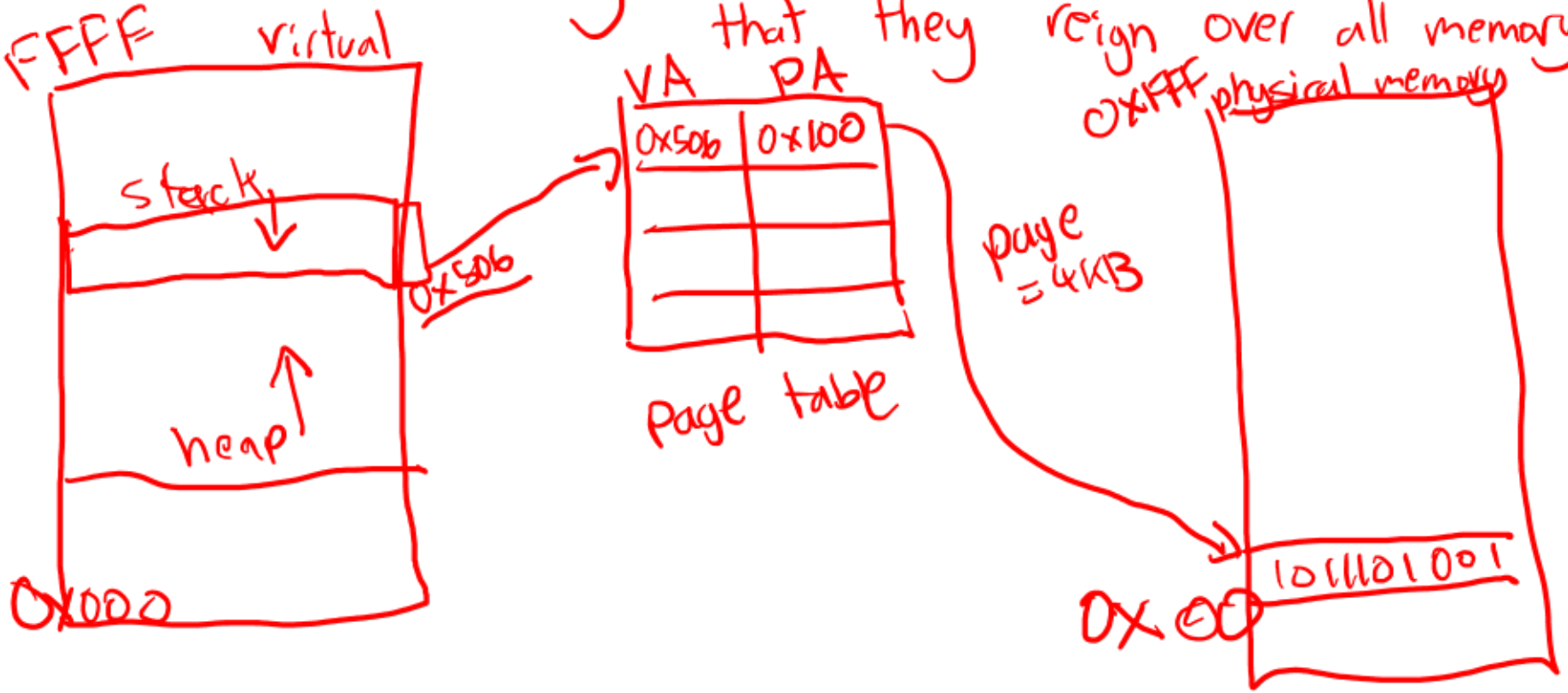
Questions/Issues

Security –

Space –

Answer/Solution

Virtual memory - abstraction given to programs that they reign over all memory



Agenda

1. A Program's Memory
    a. Description
    b. Clickers
    c. MP5 intro

1. Virtual Memory
    a. Description
    b. Examples
    c. Clickers

1. MP5 Demo

Virtual Memory

MMU - memory management unit

VA → PA - by using the page table

Virtual Memory Example #1

```
char x = 5;
printf("%x", &x);
x = x >> 3;
```

0x0FED145

1. cpu decodes the instruction:
   - load x from VA
2. MMU uses page table to find the entry for the page holding 0x0FED145
   - gives back PA (0x001)
3. search through the cache for address 0x001
4. return the bits 0000 0101 into a register
5. cpu decodes the instruction to put x and 3 into the ALU
6. store the result in a register and 0x001

Virtual Memory Example #2

Takeaways

- Virtual memory
  - Isolates programs for security and the illusion of space

- You (the programmer) cannot see or know what physical address your code is using. You can only access the virtual addresses.

- Physical memory for a program can vary in size and location.
- Virtual memory has a fixed size and a consistent location.

Am I guaranteed that ptr and ptr2 hold values 6,000 bytes apart in physical memory?

clicker.cs.illinois.edu

Q6

~Code~
340

no

```
char x = 7;
char *ptr = &x;
char *ptr2 = ptr + 6000;
```

x lives 6,000 bytes

Am I guaranteed that ptr and ptr2 hold values 6,000 bytes apart in virtual memory?

yes

```
char x = 7;

char *ptr = &x;

char *ptr2 = ptr + 6000;
```

clicker.cs.illinois.edu

Q7

~Code~
340

If program 1 stores my password in a c-string at virtual address 0xB50632 will this code in program 2 get me that password?

clicker.cs.illinois.edu

Q8

~Code~
340

```
char *ptr = (char*)0xB50632;
printf("%s", ptr);
```

NO

Is it possible for these two programs to print out the same value while running at the same time?

clicker.cs.illinois.edu

Q9

~Code~
340

```
int x = 5;

printf("%#x", &x);
```

```
int x = 2;

printf("%#x", &x);
```

Yes

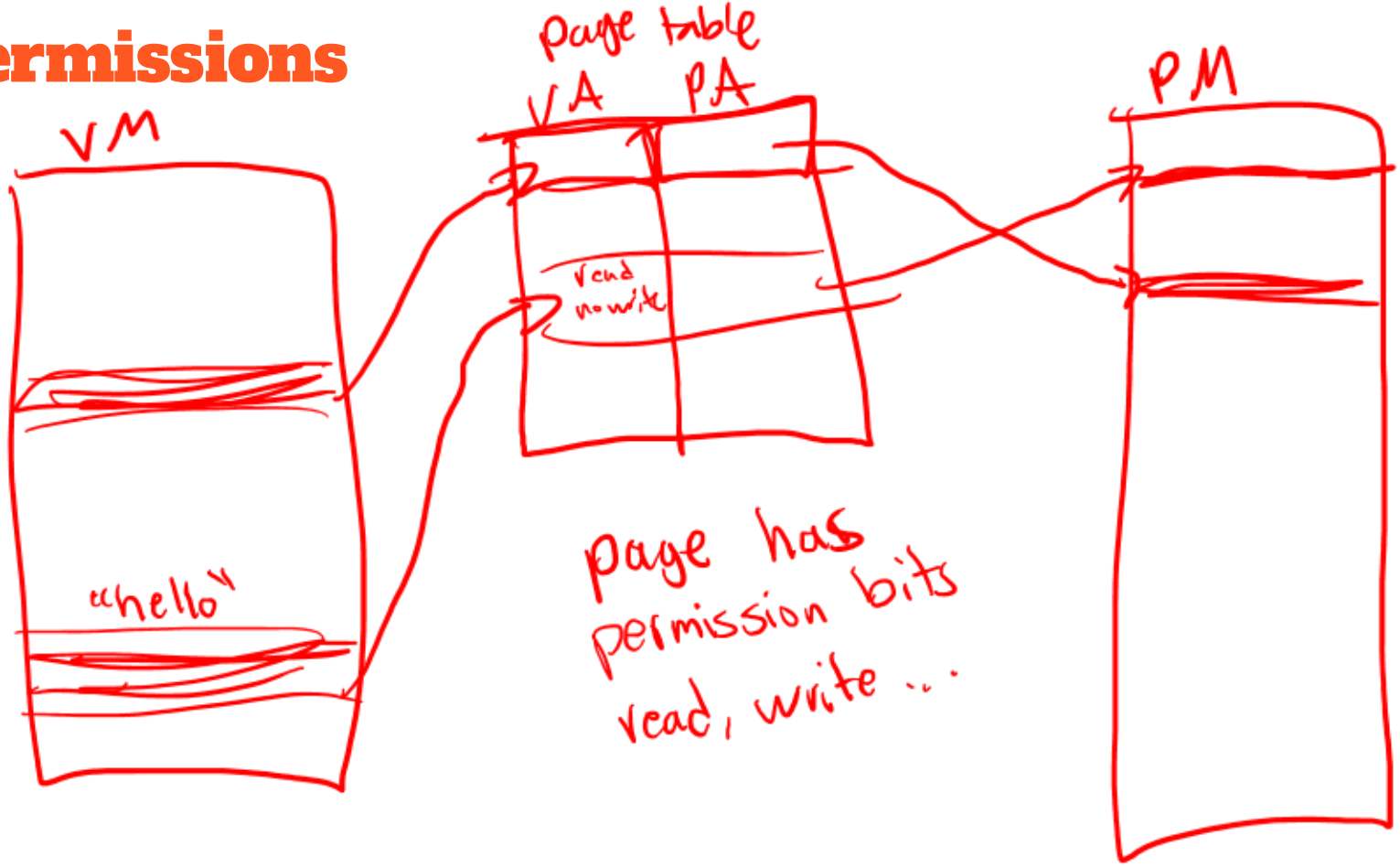
5

0x100

2

0x100

Permissions



page has permission bits read, write ...

Takeaways II

- The OS manages the page table
- The MMU reads the page table to translate addresses from virtual to physical
- Physical memory with physical address are in the cache
- Each page has permission bits that are checked at runtime

Agenda

1. A Program's Memory
    a. Description
    b. Clickers
    c. MP5 intro

1. Virtual Memory
    a. Description
    b. Examples
    c. Clickers

1. MP5 Demo

Program's Memory | Virtual Memory

Today's LGs - Deepen your understanding of how memory works in a C program and on your computer on a whole.

Be able to differentiate between globals/static, the stack, and the heap
Be able to identify common issues with the stack and the heap use
Be able to implement a simplified, efficient version of the C allocator
Be able to articulate why we use virtual memory in a computer
Be able to put together many system concepts with virtual memory